

Implementation Plan: IPTV Core Functionality

Overview

This plan implements the core IPTV desktop player features: embedded MPV playback via `mpv`, Zustand state management, electron-store persistence, typed IPC channels, multi-playlist management, category filtering, favorites, and settings configuration. Each task builds incrementally on the previous, wiring everything together at the end.

Tasks

- ☒ 1. Set up shared types, interfaces, and project dependencies
 - ☒ 1.1 Extend shared types and add new interfaces
 - Update `src/shared/types/index.ts` with extended `Channel` (add `playlistId` field), extended `Playlist` (add `url`, `addedAt` fields, rename `count` to `channelCount`), `PersistedState`, `PlayerState`, `IPCCommands`, `IPCEvents`, and `ElectronAPI` interfaces as defined in the design
 - Add error types: `NetworkError`, `TimeoutError`, `HttpError`
 - *Requirements: 1.1, 2.1, 3.1, 6.1*
 - ☒ 1.2 Install new dependencies
 - Install runtime dependencies: `zustand`, `electron-store`
 - Install dev dependencies: `fast-check`
 - *Requirements: 6.1*
- ☒ 2. Implement main process modules (StoreManager, NetworkManager, PlayerManager)
 - ☒ 2.1 Create StoreManager (`electron/store.ts`) 2.1 Create StoreManager (`electron/store.ts`)
 - Implement `StoreManager` class wrapping `electron-store` with typed schema

- Define defaults: `playlists: [], favorites: [], volume: 50, muted: false, lastActivePlaylistId: null, mpvBinaryPath: null`
- Implement `get`, `set`, `getAll`, `reset` methods
- Add schema validation: volume clamped 0-100, playlists max 50, name 1-100 chars
- Handle corrupted/missing state file gracefully (reset to defaults)
- *Requirements: 6.1, 6.2, 6.3, 6.4, 6.5*
- ☒ 2.2 Write property test for state serialization round-trip
 - **Property 6: Persisted state serialization round-trip**
 - **Validates: Requirements 6.1, 6.2**
- ☒ 2.3 Create NetworkManager (`electron/network.ts`) 2.3 Create NetworkManager (`electron/network.ts`)
 - Implement `fetchPlaylist` method using Node.js `fetch` with `AbortController` for 30-second timeout
 - Throw typed errors: `NetworkError`, `TimeoutError`, `HttpError`
 - Return raw M3U content string on success
 - *Requirements: 3.1, 3.6*
- ☒ 2.4 Rewrite PlayerManager (`electron/player.ts`) 2.4 Rewrite PlayerManager (`electron/player.ts`)
 - Remove `node-mpv` dependency, implement direct `child_process.spawn` approach
 - Spawn MPV with `--vid=<native-handle>` flag for embedded rendering
 - Use `--input-ipc-server=/tmp/mpv-iptv-socket` for JSON IPC control
 - Implement `initialize`, `play`, `stop`, `pause`, `resume`, `setVolume`, `setMute`, `getState`, `destroy` methods
 - Communicate with MPV via Unix socket (send JSON commands, parse property-change events)
 - Emit status events to renderer via `webContents.send('player:status', state)`
 - Handle MPV process crash with auto-restart logic
 - Get native window handle via `win.getNativeWindowHandle()` for X11 Window ID
 - *Requirements: 1.1, 1.2, 1.3, 1.4, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 7.5, 7.6, 7.7*

- ☒ 2.5 Write property test for volume clamping
 - **Property 5: Volume clamping invariant**
 - **Validates: Requirements 2.2, 2.7, 2.8**
- ☒ 3. Checkpoint - Ensure main process modules compile
 - Ensure all tests pass, ask the user if questions arise.
- ☒ 4. Rewrite preload bridge and IPC handlers
 - ☒ 4.1 Rewrite preload bridge (`electron/preload.ts`) 4.1 Rewrite preload bridge (`electron/preload.ts`)
 - Replace generic `ipcRenderer` exposure with typed `ElectronAPI` interface
 - Expose `player`, `playlist`, `store`, and `settings` namespaces via `contextBridge.exposeInMainWorld`
 - Implement cleanup functions for event listeners (return unsubscribe callbacks)
 - *Requirements: 1.1, 2.1, 3.1, 7.1*
 - ☒ 4.2 Rewrite IPC handlers in main process (`electron/main.ts`) 4.2 Rewrite IPC handlers in main process (`electron/main.ts`)
 - Register all `ipcMain.handle` handlers for: `player:play`, `player:stop`, `player:pause`, `player:resume`, `player:set-volume`, `player:set-mute`, `playlist:fetch`, `store:get`, `store:set`, `store:get-all`, `settings:validate-mpv-path`
 - Wire handlers to `PlayerManager`, `StoreManager`, and `NetworkManager` instances
 - Initialize `PlayerManager` with native window handle and persisted MPV path
 - Validate MPV path on startup using `fs.access` with execute permission check
 - Implement `settings:validate-mpv-path` handler that checks file existence and execute permission
 - *Requirements: 1.1, 2.1, 3.1, 6.2, 7.2, 7.3, 7.4, 7.5, 7.6, 7.7*
- ☒ 5. Implement renderer Zustand stores
 - ☒ 5.1 Create PlayerStore (`src/renderer/stores/playerStore.ts`) 5.1 Create PlayerStore (`src/renderer/stores/playerStore.ts`)

- Implement store with `status`, `currentChannel`, `volume`, `muted`, `error` state
 - Implement actions: `playChannel`, `stop`, `togglePause`, `setVolume`, `toggleMute`, `syncFromMain`
 - Actions call preload API methods and update local state
 - Subscribe to `player:status` events from main process via preload bridge
 - *Requirements: 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8*
- ☒ 5.2 Create PlaylistStore (`src/renderer/stores/playlistStore.ts`) 5.2 Create PlaylistStore (`src/renderer/stores/playlistStore.ts`)
 - Implement store with `playlists`, `activePlaylistId`, `channels`, `selectedCategory`, `categories`, `searchTerm`, `isLoading` state
 - Implement actions: `addPlaylist`, `removePlaylist`, `renamePlaylist`, `selectPlaylist`, `selectCategory`, `setSearchTerm`, `loadFromPersisted`
 - `addPlaylist` validates name (1-100 chars, non-whitespace-only), fetches M3U via preload, parses channels, persists via store API
 - `removePlaylist` also triggers orphaned favorites cleanup
 - Extract categories from channels, sort alphabetically, prepend "All"
 - *Requirements: 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7, 3.8, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6*
- ☒ 5.3 Write property test for playlist name validation
 - **Property 2: Playlist name validation rejects whitespace-only input**
 - **Validates: Requirements 3.5, 3.7**
- ☒ 5.4 Write property test for category extraction
 - **Property 3: Category extraction produces correct groupings**
 - **Validates: Requirements 4.1, 4.3, 4.4**
- ☒ 5.5 Write property test for category channel count consistency
 - **Property 8: Category channel counts are consistent**
 - **Validates: Requirements 4.6**
- ☒ 5.6 Create FavoritesStore (`src/renderer/stores/favoritesStore.ts`) 5.6 Create FavoritesStore (`src/renderer/stores/favoritesStore.ts`)
 - Implement store with `favoriteIds` (Set) state
 - Implement actions: `toggleFavorite`, `isFavorite`, `loadFromPersisted`
 - `toggleFavorite` adds if not present, removes if present (involution)

- Persist changes via store API on every toggle
 - *Requirements: 5.1, 5.2, 5.3, 5.4, 5.5, 5.6*
 - ☒ 5.7 Write property test for favorites toggle involution
 - **Property 4: Favorites toggle is an involution**
 - **Validates: Requirements 5.1, 5.2, 5.6**
 - ☒ 5.8 Create SettingsStore (`src/renderer/stores/settingsStore.ts`) 5.8 Create SettingsStore (`src/renderer/stores/settingsStore.ts`)
 - Implement store with `mpvBinaryPath`, `isValidating`, `validationError` state
 - Implement actions: `setMpvPath`, `loadFromPersisted`
 - `setMpvPath` calls `settings:validate-mpv-path` via preload, only persists if valid
 - *Requirements: 7.1, 7.2, 7.3, 7.4*
- ☒ 6. Checkpoint - Ensure stores compile and property tests pass
 - Ensure all tests pass, ask the user if questions arise.
- ☒ 7. Implement M3U parser updates and property tests
 - ☒ 7.1 Update M3U parser to support `playlistId` 7.1 Update M3U parser to support `playlistId` field
 - Modify `src/shared/utils/m3u-parser.ts` to accept a `playlistId` parameter and assign it to each parsed channel
 - Ensure channels with no `group-title` default to "Uncategorized"
 - *Requirements: 3.1, 4.1*
 - ☒ 7.2 Write property test for M3U parsing round-trip
 - **Property 1: M3U parsing round-trip preserves channel data**
 - **Validates: Requirements 3.1**
 - ☒ 7.3 Write property test for playlist removal orphan cleanup
 - **Property 7: Playlist removal cleans up orphaned favorites**
 - **Validates: Requirements 5.7**
- ☒ 8. Implement UI components

- ☒ 8.1 Create AppLayout component (`src/renderer/components/AppLayout.tsx`)
8.1 Create AppLayout component (``src/renderer/components/AppLayout.tsx``)
 - Implement top-level layout with sidebar, main content area, and player control bar
 - Use flexbox: sidebar fixed width, main content fills remaining space, control bar at bottom
 - *Requirements: 1.2, 2.1*
- ☒ 8.2 Create PlayerView, IdleView, LoadingView, ErrorView components
 - `PlayerView` (`src/renderer/components/PlayerView.tsx`): container div for embedded MPV output area, uses a ref for the native handle target
 - `IdleView` (`src/renderer/components/IdleView.tsx`): placeholder message "Select a channel to start watching"
 - `LoadingView` (`src/renderer/components/LoadingView.tsx`): spinner/skeleton while stream connects
 - `ErrorView` (`src/renderer/components/ErrorView.tsx`): error message display with retry option
 - Conditionally render based on `PlayerStore.status`
 - *Requirements: 1.2, 1.3, 1.5, 1.6*
- ☒ 8.3 Create PlayerControlBar component (`src/renderer/components/PlayerControlBar.tsx`) 8.3 Create PlayerControlBar component (``src/renderer/components/PlayerControlBar.tsx``)
 - Implement play/pause button, stop button, volume slider (0-100), mute toggle, now-playing info
 - Show control bar only when a stream is playing or paused
 - Wire buttons to PlayerStore actions
 - *Requirements: 2.1, 2.2, 2.3, 2.4, 2.5, 2.6*
- ☒ 8.4 Create VolumeSlider component (`src/renderer/components/VolumeSlider.tsx`) 8.4 Create VolumeSlider component (``src/renderer/components/VolumeSlider.tsx``)
 - Range input 0-100 with visual feedback
 - Display current volume percentage
 - Wire to `PlayerStore.setVolume`
 - *Requirements: 2.2*

- ☒ 8.5 Create Sidebar component (`src/renderer/components/Sidebar.tsx`) 8.5
Create Sidebar component (``src/renderer/components/Sidebar.tsx``)
 - Container for PlaylistSelector, CategoryList, and ChannelList
 - *Requirements: 3.3, 4.2*
- ☒ 8.6 Create PlaylistSelector component
(`src/renderer/components/PlaylistSelector.tsx`) 8.6 Create PlaylistSelector component (``src/renderer/components/PlaylistSelector.tsx``)
 - Dropdown/list to switch between playlists
 - Add playlist button (opens modal), rename (inline edit), remove (with confirmation)
 - Wire to PlaylistStore actions
 - *Requirements: 3.1, 3.3, 3.4, 3.5, 3.7, 3.8*
- ☒ 8.7 Create CategoryList component
(`src/renderer/components/CategoryList.tsx`) 8.7 Create CategoryList component (``src/renderer/components/CategoryList.tsx``)
 - Vertical list with "All" at top, then "Favorites", then alphabetically sorted categories
 - Display channel count next to each category name
 - Highlight selected category
 - Wire to PlaylistStore.selectCategory
 - *Requirements: 4.1, 4.2, 4.3, 4.4, 4.5, 4.6*
- ☒ 8.8 Update ChannelList to support favorites toggle
 - Add star icon (filled/unfilled) to each channel item
 - Wire star click to FavoritesStore.toggleFavorite
 - Show favorited channels when "Favorites" category is selected
 - *Requirements: 5.1, 5.2, 5.4, 5.5, 5.6*
- ☒ 8.9 Create SettingsDialog component
(`src/renderer/components/SettingsDialog.tsx`) 8.9 Create SettingsDialog component (``src/renderer/components/SettingsDialog.tsx``)
 - Modal with text input for MPV binary path (max 1024 chars)
 - Validate button that triggers path validation via IPC
 - Display success/error feedback

- Wire to SettingsStore actions
- *Requirements: 7.1, 7.2, 7.3, 7.4*
- ☒ 9. Checkpoint - Ensure UI components render correctly
 - Ensure all tests pass, ask the user if questions arise.
- ☒ 10. Wire everything together in App.tsx
 - ☒ 10.1 Restructure App.tsx to use new component architecture
 - Replace current App.tsx content with `AppLayout` as root
 - Initialize stores on mount: call `loadFromPersisted` on PlaylistStore, FavoritesStore, SettingsStore
 - Subscribe to player status events from main process
 - Restore last-active playlist and volume/mute state from persisted data
 - Add settings button that opens SettingsDialog
 - Add playlist modal integration with PlaylistStore.addPlaylist
 - *Requirements: 1.1, 1.6, 2.7, 2.8, 3.2, 3.3, 4.5, 5.3, 6.2, 7.1*
 - ☒ 10.2 Implement channel selection flow
 - When user clicks a channel in ChannelList, call `PlayerStore.playChannel(channel)`
 - PlayerView shows loading state, then transitions to playing when status event arrives
 - If error, show ErrorView with message
 - Switching channels stops current and starts new
 - *Requirements: 1.1, 1.3, 1.4, 1.5*
 - ☒ 10.3 Write integration tests for full IPC flow
 - Test playlist add flow: URL → fetch → parse → store → display
 - Test player lifecycle: play → status events → stop → cleanup
 - Test persistence: set state → re-read → verify restored
 - *Requirements: 1.1, 3.1, 6.1, 6.2*
- ☒ 11. Final checkpoint - Ensure all tests pass and application compiles
 - Ensure all tests pass, ask the user if questions arise.

Notes

- Tasks marked with `*` are optional and can be skipped for faster MVP
- Each task references specific requirements for traceability
- Checkpoints ensure incremental validation
- Property tests validate universal correctness properties from the design document
- Unit tests validate specific examples and edge cases
- The `node-mpv` dependency should be removed from `package.json` after task 2.4 is complete
- All IPC channels use the typed pattern defined in the design (`player:*`, `playlist:*`, `store:*`, `settings:*`)
- The M3U parser in `src/shared/utils/m3u-parser.ts` is reused and extended, not rewritten

Task Dependency Graph

```
{
  "waves": [
    { "id": 0, "tasks": ["1.1", "1.2"] },
    { "id": 1, "tasks": ["2.1", "2.3", "2.4"] },
    { "id": 2, "tasks": ["2.2", "2.5", "4.1", "7.1"] },
    { "id": 3, "tasks": ["4.2", "7.2", "7.3"] },
    { "id": 4, "tasks": ["5.1", "5.2", "5.6", "5.8"] },
    { "id": 5, "tasks": ["5.3", "5.4", "5.5", "5.7"] },
    { "id": 6, "tasks": ["8.1", "8.2", "8.3", "8.4", "8.5", "8.9"] },
    { "id": 7, "tasks": ["8.6", "8.7", "8.8"] },
    { "id": 8, "tasks": ["10.1"] },
    { "id": 9, "tasks": ["10.2", "10.3"] }
  ]
}
```